C++ COMPLETE REFERENCE LIBRARY • MODULE 03

03 OPERATORS AND EXPRESSIONS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 arithmetic relational and short circuit
3. **Theory Chapter:** 02 bitwise operators and masks
4. **Theory Chapter:** 03 precedence associativity and sequencing
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

03 — Operators and Expressions

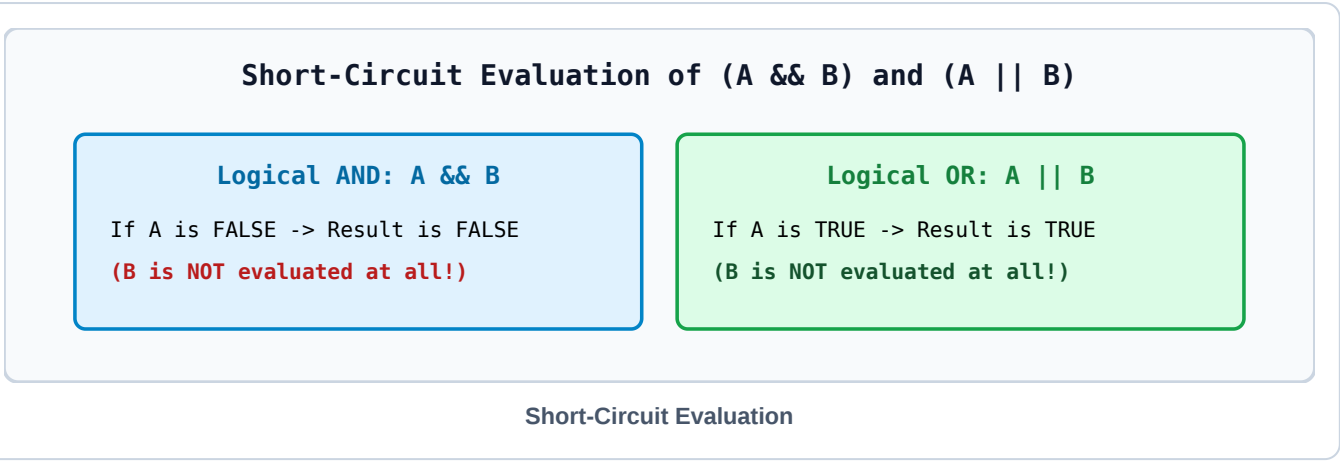
This module covers arithmetic, relational, logical (short-circuit `&&` and `||`), bitwise (`&`, `|`, `^`, `~`, `<<`, `>>`), conditional (`?:`), and comma operators, along with C++23 sequencing and evaluation-order rules.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_arithmetic_relational_and_short_circuit.md` (./01_THEORY/01_arithmetic_relational_and_short_circuit.md)
  - `02_bitwise_operators_and_masks.md` (./01_THEORY/02_bitwise_operators_and_masks.md)
  - `03_precedence_associativity_and_sequencing.md` (./01_THEORY/03_precedence_associativity_and_sequencing.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_integer_vs_floating_division.cpp` (./02_PROGRAMS/01_integer_vs_floating_division.cpp) (Category A)

  - `02_short_circuit_and_bitwise.cpp` (./02_PROGRAMS/02_short_circuit_and_bitwise.cpp) (Category A)
  - `03_unsequenced_scalar_modification_ub.cpp` (./02_PROGRAMS/03_unsequenced_scalar_modification_ub.cpp) (Category D)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_short_circuit_evaluation.svg` (./05_DIAGRAMS/diagram_01_short_circuit_evaluation.svg)

01 — Arithmetic, Relational, and Short-Circuit Logical Operators



Short-Circuit Evaluation

1. Integer Division vs. Floating-Point Division

- When **both** operands of `/` are integers (`7 / 2`), C++ performs **integer division**, truncating toward zero (`3`).
- The remainder operator `%` works **only** on integer types (`7 % 2 == 1`).

2. Short-Circuit Evaluation of `&&` and `||`

Unlike `+` or `*`, the built-in logical `&&` and `||` operators guarantee strict **Left-to-Right** evaluation with **short-circuiting**:

- In `expr1 && expr2`: If `expr1` evaluates to `false`, `expr2` is **not evaluated at all**! (This makes `if (p != nullptr && *p == 10)` completely safe!).
- In `expr1 || expr2`: If `expr1` evaluates to `true`, `expr2` is **not evaluated at all**!

02 — Bitwise Operators (`&`, `|`, `^`, `~`, `<<`, `>>`)

- `a & b` (Bitwise AND): `1` only if both bits are `1`. Useful for checking if an integer is odd: `(x & 1) ≠ 0`.
- `a | b` (Bitwise OR): `1` if at least one bit is `1`.
- `a ^ b` (Bitwise XOR): `1` if bits differ (`x ^ x == 0`, `x ^ 0 == x`).
- `x << k` (Left shift): Multiplies by 2^k (for non-negative values without overflow).
- `x >> k` (Right shift): Divides by 2^k.

03 — Operator Precedence, Associativity, and C++17/20/23 Sequencing Rules

1. **Guaranteed Sequenced Operators**:
   - Logical `&&`, `||`, ternary `?:`, and comma `,` have always sequenced their left operand before their right operand.
   - Since **C++17** (and in C++20/C++23): `a << b`, `a >> b`, `a.b`, `a→b`, and `a[b]` evaluate `a` before `b`, and assignment `a = b` evaluates `b` before `a`.
2. **Still Unsequenced in C++23**:
   - The operands of binary `+`, `-`, `*`, `/` and the separate arguments of a function call `f(a, b)` remain **unsequenced** (or indeterminately sequenced for function arguments). Never write `i = i++` or `a[i] = i++` or `i++ + ++i` — these are **Undefined Behavior**!

Visual Concept & Memory Diagrams (03 OPERATORS AND EXPRESSIONS)

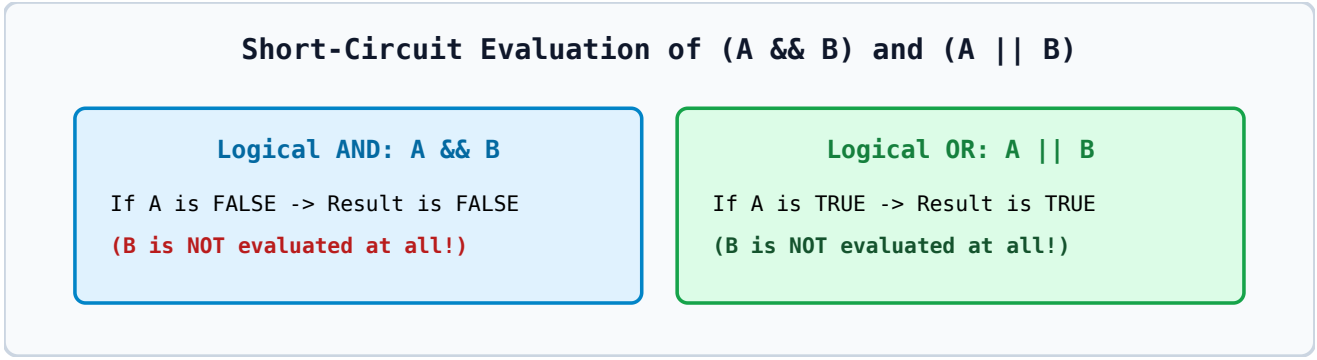


Diagram: diagram 01 short circuit evaluation

Annotated C++ Programs (03 OPERATORS AND EXPRESSIONS)

Program: `01_integer_vs_floating_division.cpp`

C++ SOURCE FILE • 01_integer_vs_floating_division.cpp

```cpp
// Program Title: Integer Division vs. Floating-Point Division and Modulo
// Topic: Operators and Expressions
// Purpose: Show truncation in 7 / 2 vs. exact division in 7.0 / 2.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_arithmetic_relational_and_short_circuit.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2;
    cout << "Integer division (7 / 2): " << (a / b) << endl;
    cout << "Remainder (7 % 2): " << (a % b) << endl;
    cout << "Floating division (7.0 / 2): " << (static_cast<double>(a) / b) << endl;
    return 0;
}

/*
Expected Deterministic Output:
Integer division (7 / 2): 3
Remainder (7 % 2): 1
Floating division (7.0 / 2): 3.5
*/
```

Program: `02_short_circuit_and_bitwise.cpp`

C++ SOURCE FILE • 02_short_circuit_and_bitwise.cpp

```cpp
// Program Title: Short-Circuit Logical Evaluation and Bitwise Operations
// Topic: Operators and Expressions — Short-Circuiting & Bitwise
// Purpose: Demonstrate that && skips its right operand when the left operand is false, plus bitwise AND/XOR.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_bitwise_operators_and_masks.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int x = 0;
    int counter = 10;

    // Because (x != 0) is false, (++counter > 0) is NEVER evaluated!
    if (x != 0 && ++counter > 0) {
        cout << "Inside if" << endl;
    }

    cout << "counter after short-circuit &&: " << counter << endl;
    cout << "Bitwise (6 & 3): " << (6 & 3) << ", (6 ^ 3): " << (6 ^ 3) << endl;
    return 0;
}

/*
Expected Deterministic Output:
counter after short-circuit &&: 10
Bitwise (6 & 3): 2, (6 ^ 3): 5
*/
```

Program: `03_unsequenced_scalar_modification_ub.cpp`

C++ SOURCE FILE • 03_unsequenced_scalar_modification_ub.cpp

```cpp
// Program Title: Unsequenced Scalar Modification Undefined Behavior (i++ + ++i)
// Topic: Operators and Expressions — Sequencing Rules
// Purpose: Explain why expressions like 'i++ + ++i' have Undefined Behavior in C++23.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/03_precedence_associativity_and_sequencing.md
// Program Classification: Category D — Undefined-behavior demonstration

#include <iostream>
using namespace std;

int main() {
    /*
    UNDEFINED BEHAVIOR EXPLANATION (Category D):
    In classroom exams, questions sometimes ask for the output of:
        int i = 5;
        int y = i++ + ++i; @@PH_6@@
    Under ISO C++ [intro.execution], binary '+' does not sequence its left and right
    operands. Modifying 'i' twice without sequencing is Undefined Behavior and has NO
    standard-guaranteed output. Always write sequenced statements instead:
    */
    int i = 5;
    int first = i++;
    int second = ++i;
    int y = first + second; // Well-defined: 5 + 7 = 12
    cout << "Well-defined sequenced result: " << y << endl;
    return 0;
}
```

Conceptual Questions — Module 03: Operators and Expressions

Q1. Why is `if (p ≠ nullptr && *p > 0)` safe even when `p == nullptr`? **Q2.** Explain why `int x = i++ + ++i;` has Undefined Behavior in C++23.

Debugging Questions — Module 03

Q4. Find the bug in `double avg = (a + b) / 2;` when `int a = 5, b = 6;`.

Predict the Output Questions — Module 03

Q3. Predict the output:

CPP

```cpp
int a = 1, b = 1;
bool res = (a == 1) || (++b == 2);
cout << res << " " << b << endl;
```

Programming Questions — Module 03

Q5. Write a C++23 program that checks whether a positive integer `n` is a power of 2 using a single bitwise expression `(n & (n - 1)) == 0`.

Detailed Solutions — Conceptual Questions (Module 03)

- **Q1 Solution**: Built-in `&&` evaluates its left operand first and short-circuits if it is `false`, so `*p` is never dereferenced when `p == nullptr`.
- **Q2 Solution**: Binary `+` does not sequence its left and right operands (`[intro.execution]`), so modifying `i` twice is unsequenced Undefined Behavior.

Detailed Solutions — Predict the Output & Debugging (Module 03)

- **Q3 Solution**: Because `(a == 1)` is `true`, `||` short-circuits and skips `(++b == 2)`, so `b` remains `1`. Output: `1 1`.
- **Q4 Solution**: `(a + b) / 2` performs integer division `11 / 2 == 5` before converting to `double` (`5.0`). Fix: `(a + b) / 2.0`.

Detailed Solutions — Programming Questions (Module 03)

CPP

```cpp
#include <iostream>
using namespace std;
int main() {
    int n = 16;
    bool is_pow2 = (n > 0) && ((n & (n - 1)) == 0);
    cout << "Is 16 power of 2? " << is_pow2 << endl;
    return 0;
}
```